**LIMKOKWING UNIVERSITY OF CREATIVE TECHNOLOGY**

Sierra Leone

**PROG101 – PRINCIPLES OF PROGRAMMING LOGIC AND DESIGN**

Individual Assignment · Department of Information and Communication Technology

# FloodAlert Salone

*Designing a Logical Solution for Flooding in Freetown: A Community Flood Early-Warning System*

| Item | Details |
|---|---|
| Assignment Title | Designing Logical Solutions for Sierra Leone Community Challenges |
| Student Name | [Your full name] |
| Student ID | [Your student ID] |
| Lecturer / Examiner | Elijah Fullah |
| Semester | 01 (September 2026 – January 2027) |
| GitHub Repository | https://github.com/[username]/ PROG101_Assignment1_[Name]_[ID] |
| Submission Date | [Week 4 date] |

# Table of Contents

**1. Rationale....3**
a) Problem Description....3
b) Background....3
c) Community Impact....3
d) Proposed Solution....4
e) Computational Thinking Process....4
f) Algorithm Design....4
g) SDG Alignment....5
h) Open Source Justification....5
i) Digital Public Goods Justification....5
**2. System Overview: Variables, Constants and Data Types....7**
**3. Algorithm (Step by Step)....7**
**4. Pseudocode....8**
**5. Flowcharts....11**
5.1 Main Program (menu with Case structure and Do–While loop)....12
5.2 Module: registerResident() (with validation loop)....12
5.3 Module: checkFloodRisk()....13
5.4 Function: calculateRisk() (multiple alternative decision with Boolean logic)....14
5.5 Module: sendAlerts() (loop with a decision inside)....15
5.6 Hierarchy Chart (Modular Design)....15
**6. Functional Modules....16**
**7. Expected Outputs (Sample Scenarios)....16**
**8. GitHub Evidence....18**
**9. Conclusion....19**
**10. References....19**

# 1. Rationale

## a) Problem Description

Every rainy season, many low-lying and hillside communities in Freetown, such as Kroo Bay, Culvert, Susan's Bay, Mabella and parts of the Regent area, experience flash floods. Water rises quickly, often at night, and residents receive little or no warning. Most people only realise that their community is in danger when water is already entering their homes. This leads to loss of life, destruction of property, displacement of families and the spread of water-borne diseases.

The problem addressed in this project is the **lack of a simple, community-level flood early-warning system** that can tell residents in good time that their area is at risk and what action they should take.

## b) Background

Sierra Leone is one of the countries most affected by climate-related disasters in West Africa. On 14 August 2017, heavy rainfall caused a landslide and floods in Freetown's Western Area that left **1,141 people dead or missing** and affected about 6,000 people (GFDRR, 2017). Floods have continued to occur in the city almost every rainy season since then.

Several factors make the problem worse. Rapid urban growth has pushed people to build in flood-prone areas close to drains, streams and the coastline. Poor waste management is another major cause: only about **21% of Freetown's waste** was estimated to be properly collected and disposed of, with the rest burned or dumped in streets and waterways, where it blocks drains and causes flooding (O'Donnell, 2019). In 2020 the Government established the **National Disaster Management Agency (NDMA)** through the National Disaster Management Agency Act (Government of Sierra Leone, 2020) to coordinate disaster preparedness and response. However, warnings still often fail to reach ordinary residents quickly. Since most households own at least a basic mobile phone, **SMS** is one of the fastest and most inclusive ways to reach them, including people without smartphones or internet access.

## c) Community Impact

The proposed system will help communities **act before a flood, not after it**. Expected impacts include:

- **Saving lives:** residents receive a warning early enough to move children, the elderly and people with disabilities to higher ground.
- **Reducing losses:** families can move documents, food and valuables in time.
- **Better coordination:** NDMA, the Freetown City Council and community disaster committees can see which communities are at risk at any moment.
- **Community participation:** trained youth volunteers record rainfall and water levels, creating local jobs and ownership.

**Beneficiaries:** residents of flood-prone communities (especially women, children, the elderly and people with disabilities), community disaster management committees, NDMA, Freetown City Council, health workers and humanitarian organisations.

**Long-term sustainability:** the system is open source, cheap to run (SMS only), uses simple manual rain gauges or low-cost sensors, and can be maintained by local developers and university students. Ownership can be handed to NDMA and the city council, and the software can be reused in other districts such as Kenema, Bo and Makeni.

## d) Proposed Solution

The proposed solution is **FloodAlert Salone**, a community flood early-warning system. Its main functions are:

1. **Register residents** with their name, phone number and community.
2. **Record readings:** a trained volunteer (or a low-cost sensor) enters the rainfall in the last hour (mm) and the water level of the main drain or river (m) for a community.
3. **Calculate the flood risk level** using clear rules: GREEN (normal), YELLOW (stay alert), ORANGE (prepare to move) or RED (evacuate now).
4. **Send SMS alerts** with simple instructions to every registered resident of that community when the risk is YELLOW, ORANGE or RED.
5. **Produce a report** showing the number of residents, checks made and alerts sent.

The thresholds used in this design (30 mm/hour heavy rain, 50 mm/hour extreme rain, 1.5 m warning level and 2.5 m danger level) are illustrative values and are stored as **constants**, so they can easily be adjusted by NDMA and the Sierra Leone Meteorological Agency based on local data.

## e) Computational Thinking Process

| Pillar | How it was applied |
|---|---|
| **Decomposition** | The big problem "warn people about floods" was broken into smaller tasks: register residents, collect readings, validate readings, calculate risk, prepare a message, send alerts and produce a report. Each task became a module or function. |
| **Pattern recognition** | Floods follow a pattern: they happen after heavy rainfall and/or when drains and rivers are already full, and the same communities are affected repeatedly. This pattern gave the rules for the risk levels. |
| **Abstraction** | Only the important details were kept: rainfall (mm), water level (m), community name and residents' phone numbers. Details such as house type or ages were left out because they do not affect the warning. |
| **Algorithm design** | Clear, ordered steps were written as an algorithm, pseudocode and flowcharts, using decision structures, loops, modules and functions (sections 2–5). |

## f) Algorithm Design

The system follows the **Input → Processing → Output** model:

| Stage | Description |
|---|---|
| **Input** | Menu choice; resident name, phone and community; community name, rainfall (mm) and water level (m). |
| **Processing** | Validate the inputs (no negative values, 9-digit phone numbers); compare rainfall and water level against constants using Boolean logic; choose a risk level; select the matching message; loop through all residents to find those in the affected community; count alerts. |
| **Output** | Risk level and safety message on screen; SMS alerts to residents; confirmation messages; summary report. |

The design uses: **sequence** (steps in order), **decision structures** (If–Then–Else If, nested If and Select Case), **Boolean logic** (AND, OR), **repetition structures** (Do–While menu loop, While validation loops and For loops), **modules** (displayMenu, registerResident, checkFloodRisk, sendAlerts, viewReport) and **functions** (calculateRisk and getMessage, which return values).

## g) SDG Alignment

- **SDG 11 – Sustainable Cities and Communities, Target 11.5:** significantly reduce the number of deaths and people affected by disasters, including water-related disasters (United Nations, 2015). FloodAlert Salone directly aims to reduce flood deaths and losses in Freetown's communities.
- **SDG 13 – Climate Action, Target 13.1:** strengthen resilience and adaptive capacity to climate-related hazards. Early warning is one of the most cost-effective ways to adapt to heavier rainfall caused by climate change.
- **SDG 3 – Good Health and Well-being (supporting):** fewer flood victims and less exposure to dirty floodwater reduce injuries and diseases such as cholera and typhoid.

## h) Open Source Justification

FloodAlert Salone should be **open source** (released under the **MIT licence**) because:

- **It is a public safety tool.** Lives should not depend on whether a council can afford licence fees.
- **Benefits for Sierra Leone:** no licence costs; local developers can adapt it (for example adding messages in Krio, Temne or Mende, or voice calls for people who cannot read); there is no dependence on a single foreign vendor; and students at Limkokwing and other universities can learn from and improve real code.
- **Potential community contributions:** university students and developers can add features (sensor integration, maps, a mobile app); NGOs can translate messages; meteorologists can improve the risk rules; and communities can report errors through GitHub "issues". Every improvement made in one district can be shared with all others.

## i) Digital Public Goods Justification

A **Digital Public Good (DPG)** is open-source software, open data or open content that respects privacy and helps achieve the SDGs. FloodAlert Salone could be registered with the **Digital Public Goods Alliance** because it can meet the nine indicators of the DPG Standard (DPGA, 2026):

| DPG indicator | How FloodAlert Salone meets it |
|---|---|
| 1. SDG relevance | Supports SDG 11.5, SDG 13.1 and SDG 3 |
| 2. Open licence | Released under the MIT licence on GitHub |
| 3. Clear ownership | Owned by the project team, with ownership later transferred to NDMA / a public body |
| 4. Platform independence | Works with any SMS gateway and any computer; no paid lock-in |
| 5. Documentation | README, pseudocode, flowcharts and user guide in the repository |
| 6. Non-personal data extraction | Rainfall and risk-level data can be exported as open data (e.g. CSV) without personal details |
| 7. Privacy & applicable laws | Phone numbers stored only with consent, used only for alerts, never |

| DPG indicator | How FloodAlert Salone meets it |
|---|---|
| | published |
| 8. Standards & best practices | Clear modular design, version control, standard formats |
| 9. Do no harm by design | Data kept secure, no inappropriate content, messages tested to avoid false panic |

The system promotes **accessibility** (basic SMS works on any phone), **openness** (anyone can see and improve the code) and **public benefit** (free early warning for the most vulnerable). Similar tools already exist as DPGs: for example, the **Ushahidi** crisis-mapping platform is a recognised DPG used in disasters (DPGA, 2024), and Sierra Leone already uses the open-source **DHIS2** platform for national health information (Kinkade *et al.*, 2022). This shows that open digital tools can work at national scale in Sierra Leone.

## 2. System Overview: Variables, Constants and Data Types

| Name | Data type | Purpose |
|---|---|---|
| `HEAVY_RAIN` (constant = 30.0) | Real | Rainfall (mm/hour) that counts as heavy |
| `EXTREME_RAIN` (constant = 50.0) | Real | Rainfall (mm/hour) that counts as extreme |
| `WARNING_LEVEL` (constant = 1.5) | Real | Water level (m) that needs attention |
| `DANGER_LEVEL` (constant = 2.5) | Real | Water level (m) that is dangerous |
| `MAX_RESIDENTS` (constant = 500) | Integer | Size of the resident arrays |
| `residentName[ ]` | String array | Names of registered residents |
| `residentPhone[ ]` | String array | Phone numbers, e.g. "076123456" |
| `residentCommunity[ ]` | String array | Community of each resident |
| `residentCount` | Integer | Number of registered residents |
| `totalChecks`, `totalAlertsSent` | Integer | Counters for the report |
| `choice` | Integer | Menu option chosen by the user |
| `community` | String | Community being checked |
| `rainfall` | Real | Rainfall in the last hour (mm) |
| `riverLevel` | Real | Water level in the drain/river (m) |
| `risk` | String | "GREEN", "YELLOW", "ORANGE" or "RED" |
| `index`, `sent` | Integer | Loop counter and number of SMS sent |

*Note:* phone numbers are stored as **Strings** because they begin with 0 and are not used in calculations. Sierra Leone mobile numbers written locally have 9 digits (e.g. 076123456).

## 3. Algorithm (Step by Step)

**Main program**

1. Start.
2. Set residentCount, totalChecks and totalAlertsSent to 0.
3. Display the main menu: 1 Register resident, 2 Check flood risk, 3 View report, 4 Exit.
4. Input the user's choice.
5. If choice is 1, go to the *Register Resident* steps. If 2, go to *Check Flood Risk*. If 3, go to *View Report*. If 4, display "Goodbye" and go to step 7. Otherwise display "Invalid choice".
6. Go back to step 3 (repeat until the choice is 4).
7. Stop.

**Register Resident**

1. Input the resident's name.
2. Input the phone number. While the phone number does not have 9 digits, display an error and input it again.
3. Input the community.
4. Store the name, phone and community at position residentCount in the arrays.
5. Add 1 to residentCount and display "Resident registered".

**Check Flood Risk**

1. Input the community name.
2. Input the rainfall. While rainfall is less than 0, display an error and input it again.
3. Input the water level. While the water level is less than 0, display an error and input it again.
4. Calculate the risk level:
    a) If rainfall ≥ 50 OR water level ≥ 2.5, risk = RED.
    b) Else if rainfall ≥ 30 AND water level ≥ 1.5, risk = ORANGE.
    c) Else if rainfall ≥ 30 OR water level ≥ 1.5, risk = YELLOW.
    d) Otherwise, risk = GREEN.
5. Add 1 to totalChecks. Display the community, risk level and the matching safety message.
6. If risk is GREEN, display "No alert needed". Otherwise, send alerts.

**Send Alerts**

1. Set sent to 0 and get the message for the risk level.
2. For each resident from the first to the last: if the resident's community equals the community being checked, send the SMS to the resident's phone and add 1 to sent.
3. Add sent to totalAlertsSent and display how many residents were alerted.

**View Report**

1. Display the number of registered residents, total checks and total alerts sent.
2. For each resident, display their number, name and community.

## 4. Pseudocode

```
// ========================================================
// FloodAlert Salone - Community Flood Early-Warning System
// PROG101 Assignment - Pseudocode (Gaddis style)
// ========================================================

// ---------- Global constants ----------
Constant Real HEAVY_RAIN = 30.0       // mm of rain in one hour
Constant Real EXTREME_RAIN = 50.0     // mm of rain in one hour
Constant Real WARNING_LEVEL = 1.5     // drain/river water level (m)
Constant Real DANGER_LEVEL = 2.5      // drain/river water level (m)
Constant Integer MAX_RESIDENTS = 500

// ---------- Global variables (shared by modules) ----------
Declare String residentName[MAX_RESIDENTS]
Declare String residentPhone[MAX_RESIDENTS]
Declare String residentCommunity[MAX_RESIDENTS]
Declare Integer residentCount = 0
```

```
Declare Integer totalChecks = 0
Declare Integer totalAlertsSent = 0

// ---------- Main module ----------
Module main()
   Declare Integer choice
   Do
      Call displayMenu()
      Display "Enter your choice (1-4): "
      Input choice
      Select choice
         Case 1:
            Call registerResident()
         Case 2:
            Call checkFloodRisk()
         Case 3:
            Call viewReport()
         Case 4:
            Display "Goodbye. Stay safe!"
         Default:
            Display "Invalid choice. Please enter 1 to 4."
      End Select
   While choice != 4
End Module

// ---------- Display the menu ----------
Module displayMenu()
   Display "===== FloodAlert Salone ====="
   Display "1. Register resident"
   Display "2. Check flood risk and send alerts"
   Display "3. View report"
   Display "4. Exit"
End Module

// ---------- Register a resident ----------
Module registerResident()
   Declare String name, phone, community
   Display "Enter resident name: "
   Input name
   Display "Enter phone number (9 digits, e.g. 076123456): "
   Input phone
   While length(phone) != 9              // validation loop
      Display "Phone must have 9 digits. Re-enter: "
      Input phone
   End While
   Display "Enter community (e.g. Kroo Bay): "
   Input community
   Set residentName[residentCount] = name
   Set residentPhone[residentCount] = phone
   Set residentCommunity[residentCount] = community
   Set residentCount = residentCount + 1
   Display "Resident registered. Total residents: ", residentCount
End Module

// ---------- Check flood risk for a community ----------
Module checkFloodRisk()
   Declare String community, risk
```

```
   Declare Real rainfall, riverLevel
   Display "Enter community name: "
   Input community
   Display "Enter rainfall in the last hour (mm): "
   Input rainfall
   While rainfall < 0                    // validation loop
      Display "Invalid - rainfall cannot be negative. Re-enter: "
      Input rainfall
   End While
   Display "Enter drain/river water level (m): "
   Input riverLevel
   While riverLevel < 0                  // validation loop
      Display "Invalid - water level cannot be negative. Re-enter: "
      Input riverLevel
   End While
   Set risk = calculateRisk(rainfall, riverLevel)   // function call
   Set totalChecks = totalChecks + 1
   Display "Community: ", community
   Display "Risk level: ", risk
   Display getMessage(risk)
   If risk == "GREEN" Then
      Display "No alert needed."
   Else
      Call sendAlerts(community, risk)
   End If
End Module

// ---------- FUNCTION: work out the risk level ----------
Function String calculateRisk(Real rainfall, Real riverLevel)
   Declare String risk
   If rainfall >= EXTREME_RAIN OR riverLevel >= DANGER_LEVEL Then
      Set risk = "RED"
   Else If rainfall >= HEAVY_RAIN AND riverLevel >= WARNING_LEVEL Then
      Set risk = "ORANGE"
   Else If rainfall >= HEAVY_RAIN OR riverLevel >= WARNING_LEVEL Then
      Set risk = "YELLOW"
   Else
      Set risk = "GREEN"
   End If
   Return risk
End Function

// ---------- FUNCTION: choose the safety message ----------
Function String getMessage(String risk)
   Declare String message
   Select risk
      Case "RED":
         Set message = "RED ALERT: Flooding expected. Move to higher ground NOW
and follow NDMA instructions."
      Case "ORANGE":
         Set message = "ORANGE WARNING: High flood risk. Prepare to move. Keep
children away from drains."
      Case "YELLOW":
         Set message = "YELLOW ALERT: Heavy rain or rising water. Stay alert and
clear your drains."
      Default:
         Set message = "GREEN: Normal conditions."
```

```
   End Select
   Return message
End Function

// ---------- Send SMS to residents of one community ----------
Module sendAlerts(String community, String risk)
   Declare Integer index
   Declare Integer sent = 0
   Declare String message = getMessage(risk)
   For index = 0 To residentCount - 1           // count-controlled loop
      If residentCommunity[index] == community Then   // decision inside loop
         Call sendSMS(residentPhone[index], message)  // provided by the SMS
gateway
         Set sent = sent + 1
      End If
   End For
   Set totalAlertsSent = totalAlertsSent + sent
   Display sent, " residents alerted in ", community
End Module

// ---------- Summary report ----------
Module viewReport()
   Declare Integer index
   Display "===== FloodAlert Salone Report ====="
   Display "Registered residents: ", residentCount
   Display "Flood checks done: ", totalChecks
   Display "SMS alerts sent: ", totalAlertsSent
   For index = 0 To residentCount - 1
      Display index + 1, ". ", residentName[index], " - ",
residentCommunity[index]
   End For
End Module
```

## 5. Flowcharts

The flowcharts below use the standard symbols: ovals (start/end), parallelograms (input/output), rectangles (processes), diamonds (decisions) and double-sided boxes (calls to modules). They were drawn to match the pseudocode exactly and can be recreated in diagrams.net (Draw.io).

## 5.1 Main Program (menu with Case structure and Do–While loop)

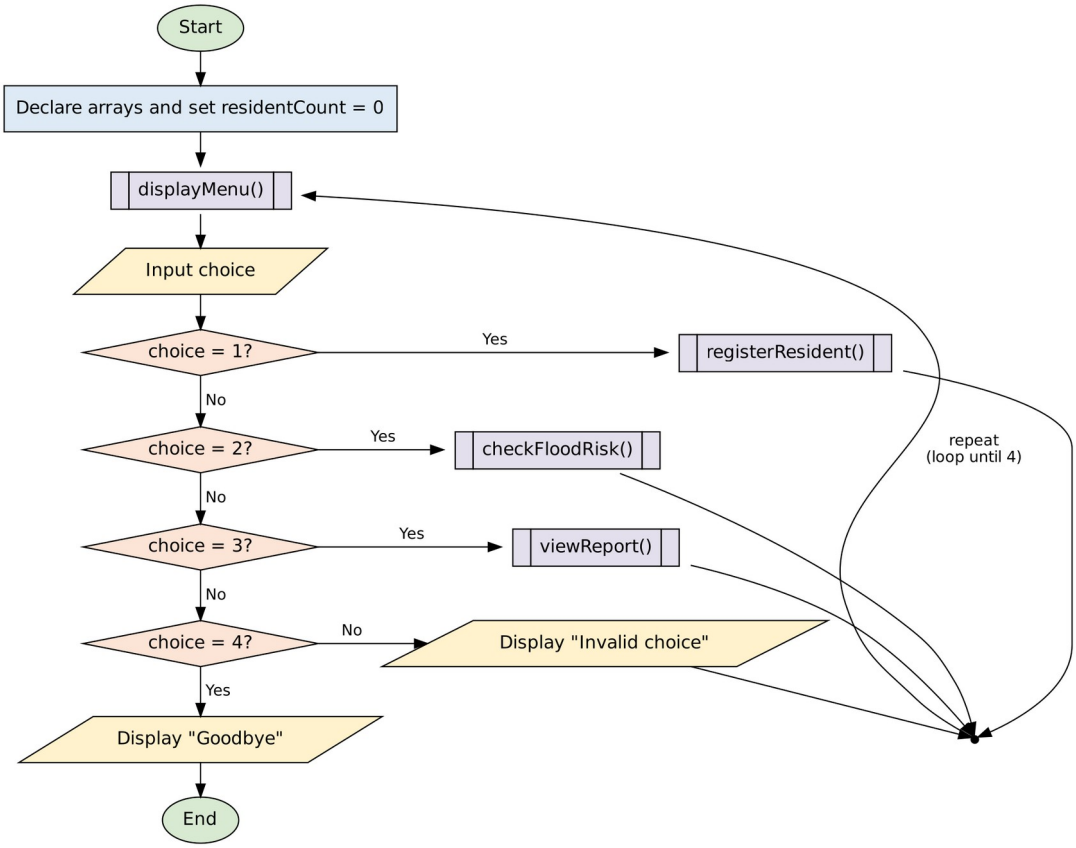



*Figure 1: Main program flowchart*

## 5.2 Module: registerResident() (with validation loop)

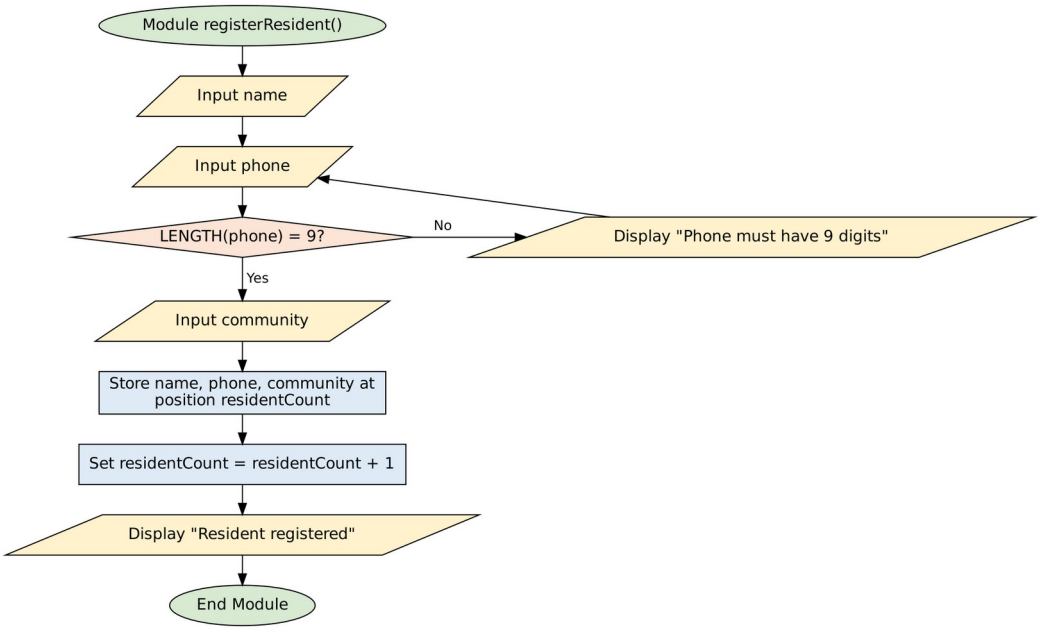



*Figure 2: Register resident flowchart*

## 5.3 Module: checkFloodRisk()

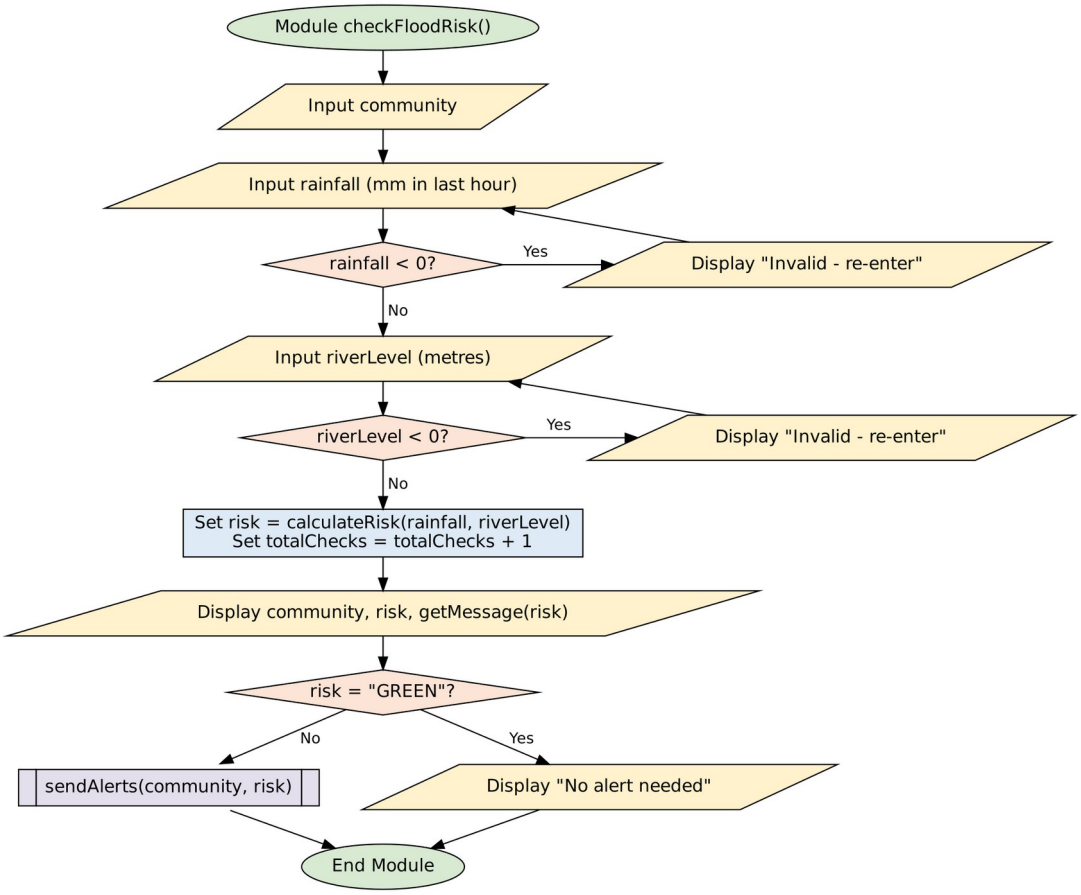



*Figure 3: Check flood risk flowchart*

## 5.4 Function: calculateRisk() (multiple alternative decision with Boolean logic)

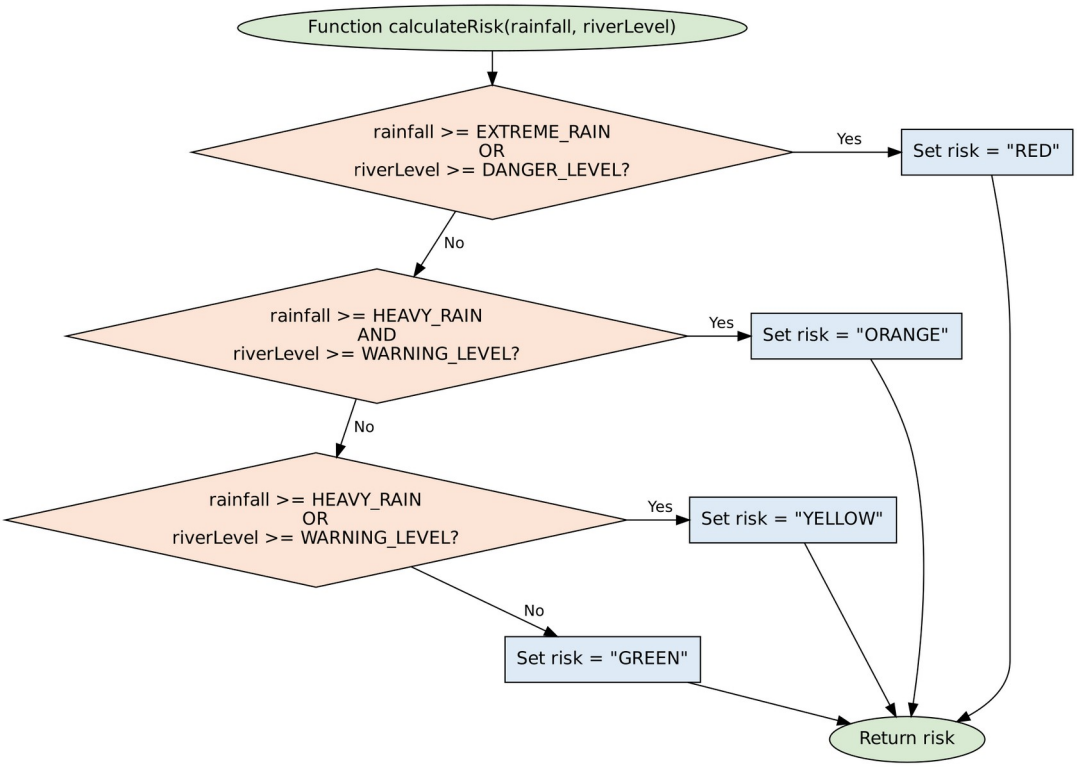



*Figure 4: calculateRisk function flowchart*

## 5.5 Module: sendAlerts() (loop with a decision inside)

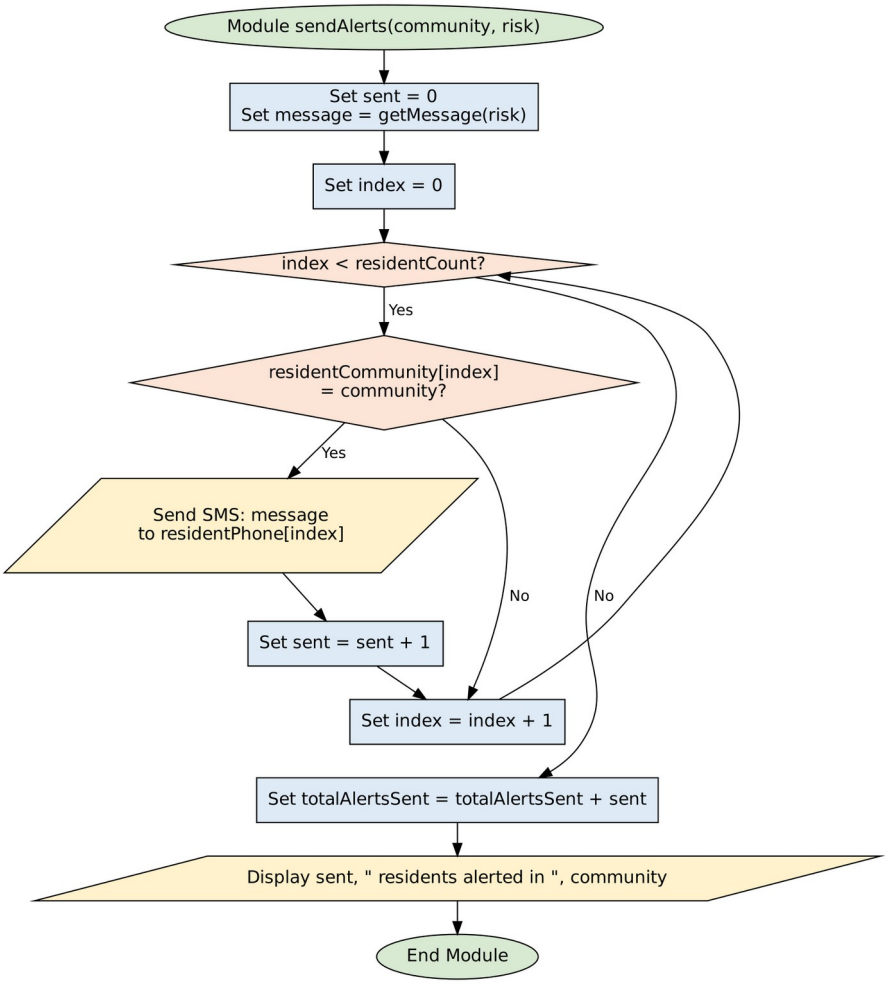



*Figure 5: sendAlerts module flowchart*

## 5.6 Hierarchy Chart (Modular Design)

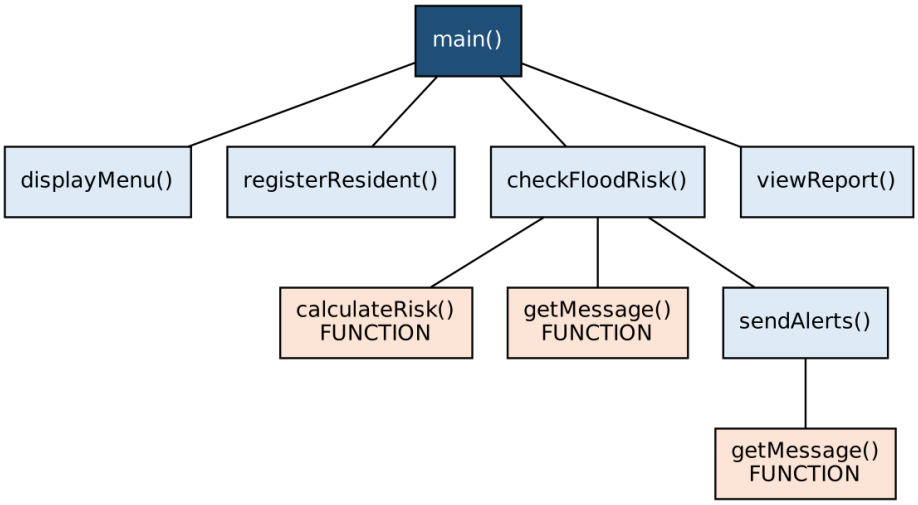



*Figure 6: Hierarchy chart showing how modules call each other*

## 6. Functional Modules

| Module / Function | Purpose | Parameters → Returns | Logic structures used |
|---|---|---|---|
| `main()` | Controls the program and menu | None | Do–While loop (post-test), Select Case |
| `displayMenu()` | Shows the menu options | None | Sequence |
| `registerResident()` | Adds a new resident | None | While validation loop, arrays |
| `checkFloodRisk()` | Collects readings, finds risk, triggers alerts | None | While validation loops, If–Then–Else, function calls |
| `calculateRisk()` **function** | Decides the risk level | rainfall, riverLevel (Real) → risk (String) | If–Then–Else If, Boolean AND / OR |
| `getMessage()` **function** | Chooses the safety message | risk (String) → message (String) | Select Case |
| `sendAlerts()` | Sends SMS to residents of a community | community, risk (String) | For loop (count-controlled), If inside loop, counter |
| `viewReport()` | Shows summary statistics | None | For loop, sequence |

**Where each required logic element appears:**

- **Decision structures:** simple/dual (`If risk == "GREEN"`), multiple alternative (`calculateRisk`), decision nested inside a loop (`sendAlerts`), case structure (`main`, `getMessage`).
- **Boolean logic:** `OR` for RED and YELLOW, `AND` for ORANGE.
- **Repetition:** post-test Do–While (menu), condition-controlled While (input validation), count-controlled For (sending alerts, report).
- **Modules and functions:** six modules and two value-returning functions, with parameters and arguments.
- **Variables, constants and data types:** see section 2.

## 7. Expected Outputs (Sample Scenarios)

Assume these residents are registered: **Aminata Kamara (Kroo Bay)**, **Mohamed Sesay (Culvert)**, **Fatmata Conteh (Kroo Bay)**, **Ibrahim Bangura (Kroo Bay)** and **Mariama Koroma (Mabella)**.

| No. | Community | Rainfall (mm) | Level (m) | Risk | Expected output |
|---|---|---|---|---|---|
| 1 | Kroo Bay | 12 | 0.8 | GREEN | "GREEN: Normal conditions." "No alert needed." |
| 2 | Culvert | 35 | 1.2 | YELLOW | Rain ≥ 30 is True but level ≥ 1.5 is False, so AND fails and OR passes. 1 SMS sent (Mohamed). |
| 3 | Kroo Bay | 42 | 1.8 | ORANGE | Not RED (42 < 50, 1.8 < 2.5). Both |

| No. | Community | Rainfall (mm) | Level (m) | Risk | Expected output |
|---|---|---|---|---|---|
| | | | | | heavy-rain and warning-level are True, so ORANGE. **3 SMS** sent. |
| 4 | Mabella | 55 | 1.0 | RED | Rainfall ≥ 50, so RED (OR needs only one True). 1 SMS sent (Mariama). |
| 5 | Susan's Bay | 20 | 2.7 | RED | Water level ≥ 2.5, so RED. 0 SMS (no registered residents there yet). |
| 6 | Kroo Bay | −5 | – | – | "Invalid - rainfall cannot be negative. Re-enter:" (validation loop) |

**Sample screen output for scenario 3:**

```
===== FloodAlert Salone =====
1. Register resident
2. Check flood risk and send alerts
3. View report
4. Exit
Enter your choice (1-4): 2
Enter community name: Kroo Bay
Enter rainfall in the last hour (mm): 42
Enter drain/river water level (m): 1.8
Community: Kroo Bay
Risk level: ORANGE
ORANGE WARNING: High flood risk. Prepare to move. Keep children away from
drains.
3 residents alerted in Kroo Bay
```

**Dry run (trace table) of sendAlerts("Kroo Bay", "ORANGE") with residentCount = 5:**

| index | residentCommunity[index] | = "Kroo Bay"? | sent |
|---|---|---|---|
| 0 | Kroo Bay | True → SMS to Aminata | 1 |
| 1 | Culvert | False | 1 |
| 2 | Kroo Bay | True → SMS to Fatmata | 2 |
| 3 | Kroo Bay | True → SMS to Ibrahim | 3 |
| 4 | Mabella | False | 3 |
| 5 | (loop ends: index > residentCount − 1) | – | **3** |

**Boundary test of calculateRisk:**

| rainfall | riverLevel | Result and reason |
|---|---|---|
| 30.0 | 1.5 | ORANGE: both values are exactly on the limit and >= makes both True |
| 29.9 | 1.4 | GREEN: every condition is False |

| rainfall | riverLevel | Result and reason |
|---|---|---|
| 50.0 | 0.0 | RED: rainfall equals EXTREME_RAIN |

## 8. GitHub Evidence

| Item | Details |
|---|---|
| Repository name | `PROG101_Assignment1_[StudentName]_[StudentID]` |
| Repository URL | https://github.com/[your-username]/ PROG101_Assignment1_[StudentName]_[StudentID] |

**Repository contents:**

| File | Description |
|---|---|
| `README.md` | Project overview (content below) |
| `Report/FloodAlert_Salone_Report.pdf` | This report |
| `Pseudocode/FloodAlert_Pseudocode.md` | Full pseudocode (section 4) |
| `Flowcharts/*.png` | Main, register, check risk, calculateRisk, sendAlerts, hierarchy chart |
| `LICENSE` | MIT licence |

**Commit history (planned regular commits):**

| Day | Commit message | Files |
|---|---|---|
| Day 1 | Initial commit: add README and MIT licence | README.md, LICENSE |
| Day 2 | Add problem analysis and rationale draft | Report draft |
| Day 3 | Add algorithm and pseudocode | Pseudocode file |
| Day 4 | Add flowcharts exported from draw.io | Flowcharts folder |
| Day 5 | Add expected outputs and trace tables | Report update |
| Day 6 | Add final report PDF | Final report |

**[Insert Screenshot 1: repository home page showing files and README]**

**[Insert Screenshot 2: commit history page showing dates and messages]**

**[Insert Screenshot 3: README displayed on GitHub]**

**README.md content:**

```
# FloodAlert Salone
A community flood early-warning system for Freetown, Sierra Leone.
PROG101 Principles of Programming Logic and Design - Assignment 1
Student: [Your Name] | ID: [Student ID] | Lecturer: Elijah Fullah
```

```
## Problem
Flash floods hit communities such as Kroo Bay, Culvert and Mabella
every rainy season, and residents get little or no warning.

## Solution
Volunteers enter rainfall (mm) and drain/river level (m). The system
calculates a risk level (GREEN, YELLOW, ORANGE, RED) and sends SMS
alerts with safety instructions to registered residents.

## Contents
- Report/ - full design report (PDF)
- Pseudocode/ - complete pseudocode
- Flowcharts/ - flowcharts drawn in diagrams.net

## Logic used
Variables, constants, data types, decision structures, Boolean logic
(AND/OR), repetition structures, modules, functions and arrays.

## SDGs
SDG 11 (Target 11.5) and SDG 13 (Target 13.1).

## Licence
MIT - free to use, study, change and share (Digital Public Good ready).
```

## 9. Conclusion

FloodAlert Salone shows how computational thinking and programming logic can be used to design a practical solution to one of Sierra Leone's most serious problems. By breaking the problem down into modules, using constants for flood thresholds, applying decision structures with Boolean logic to calculate risk levels, and using loops to validate input and send alerts to many residents, the design is clear, efficient and easy to turn into a real program. Because it is planned as open-source software that meets the Digital Public Goods Standard, it can be freely adopted, improved and scaled by NDMA, city councils and local developers, contributing to SDG 11 and SDG 13 and helping to save lives in flood-prone communities.

## 10. References

Chacon, S. and Straub, B. (2014) *Pro Git*. 2nd edn. Berkeley, CA: Apress. Available at: https://git-scm.com/book (Accessed: 15 October 2026).

Digital Public Goods Alliance (2024) *The Ushahidi Platform*. DPG Registry. Available at: https://www.digitalpublicgoods.net/r/ushahidi (Accessed: 15 October 2026).

Digital Public Goods Alliance (2026) *Digital Public Goods Standard*. Available at: https://www.digitalpublicgoods.net/standard (Accessed: 15 October 2026).

Gaddis, T. (2023) *Starting Out with Programming Logic and Design*. 6th edn. Hoboken, NJ: Pearson.

GFDRR (2017) *Sierra Leone: Rapid Damage and Loss Assessment of August 14th, 2017 Landslides and Floods in the Western Area*. Washington, DC: World Bank / Global Facility for Disaster Reduction and Recovery.

Government of Sierra Leone (2020) *The National Disaster Management Agency Act, 2020* (Act No. 3 of 2020). Freetown: Government Printer.

Kinkade, C. *et al.* (2022) 'Extending and strengthening routine DHIS2 surveillance systems for COVID-19 responses in Sierra Leone, Sri Lanka, and Uganda', *Emerging Infectious Diseases*, 28(Suppl. 1), pp. S42–S48.

O'Donnell, E. (2019) *Sierra Leone's waste and flooding crisis*. Concern Worldwide. Available at: https://concern.net/news/sierra-leones-waste-and-flooding-crisis (Accessed: 15 October 2026).

United Nations (2015) *Transforming our world: the 2030 Agenda for Sustainable Development* (A/RES/70/1). New York: United Nations.

Wing, J.M. (2006) 'Computational thinking', *Communications of the ACM*, 49(3), pp. 33–35.